

# Requisiti progetto rubricaJavaAngular

## Obiettivo

Creare la base di un'applicazione full stack per la gestione di una rubrica personale multiutente.

Il nome ufficiale del progetto, sia in locale sia nel repository GitHub, deve essere:

rubricaJavaAngular

L'applicazione deve essere composta da:

- \* frontend Angular per interfaccia grafica, login e gestione contatti;
- \* backend Spring Boot con Gradle;
- \* autenticazione gestita con Spring Security;
- \* database locale per utenti, contatti e dati collegati alla rubrica;
- \* validazione e sanitizzazione dei dati lato frontend, backend e database.

## Architettura generale

La struttura del repository dovrà essere organizzata in due moduli principali:

frontend/  
backend/

In sviluppo:

Frontend Angular: <http://localhost:4200>

Backend Spring Boot: <http://localhost:8080>

In una possibile configurazione integrata, Spring Boot potrà servire il build statico Angular e l'applicazione sarà raggiungibile dal path base del backend.

Il progetto deve supportare due modalità operative:

- \* modalità sviluppo, con frontend Angular e backend Spring Boot avviati separatamente;
- \* modalità produzione/integrata, con build statica Angular servita direttamente dal backend Spring Boot.

## Integrazione build Angular nel backend

In modalità produzione/integrata, la build statica del frontend Angular deve essere

inserita nel backend Spring Boot e servita come risorsa statica.

Flusso previsto:

```
cd frontend  
npm run build
```

Output Angular previsto:

```
frontend/dist/<nome-app>/
```

I file generati dalla build Angular devono essere copiati nel backend in una posizione standard Spring Boot:

```
backend/src/main/resources/static/
```

In questo modo Spring Boot potrà servire l'app Angular dal path base:

```
http://localhost:8080/
```

Le API REST devono rimanere sotto path dedicato:

```
/api/**
```

Requisiti:

- \* la build Angular non deve essere committata se considerata artifact generato, salvo scelta esplicita documentata;
- \* il processo di build deve documentare come generare e copiare gli artifact Angular nel backend;
- \* il processo può essere automatizzato tramite script npm, task Gradle o pipeline CI;
- \* Spring Boot deve servire correttamente index.html e gli asset statici Angular;
- \* le rotte Angular come /login, /home, /error e /contacts devono funzionare anche con refresh diretto del browser;
- \* il backend deve prevedere un fallback SPA che inoltri le rotte frontend a index.html;
- \* il fallback SPA non deve intercettare le API /api/\*\*;
- \* in sviluppo devono essere configurati CORS o proxy Angular per permettere la comunicazione tra localhost:4200 e localhost:8080;
- \* in produzione/integrata Angular deve chiamare le API usando path relativi, ad esempio /api/auth/login.

## Distinzione tra sviluppo e produzione

Il progetto deve distinguere chiaramente tra ambiente di sviluppo e ambiente di produzione.

La distinzione deve essere gestita sia lato backend Spring Boot sia lato frontend Angular.

## Backend Spring Boot

Il backend deve usare profili Spring dedicati.

File consigliati:

backend/src/main/resources/application.yml

backend/src/main/resources/application-dev.yml

backend/src/main/resources/application-prod.yml

Requisiti:

- \* usare dev come profilo per lo sviluppo locale;
- \* usare prod come profilo per produzione o build integrata;
- \* definire in application.yml le impostazioni comuni;
- \* definire in application-dev.yml configurazioni locali e piu' permissive;
- \* definire in application-prod.yml configurazioni piu' restrittive;
- \* evitare credenziali reali committate nei file di configurazione;
- \* usare variabili d'ambiente o configurazioni esterne per segreti e impostazioni sensibili;
- \* documentare il profilo attivo e il comando di avvio.

Configurazioni tipiche per dev:

- \* CORS abilitato per http://localhost:4200;
- \* database locale H2;
- \* console H2 abilitata solo se utile allo sviluppo;
- \* logging piu' dettagliato, utile al debug e alla diagnosi locale;
- \* eventuali dati demo;
- \* errori tecnici loggati in modo piu' verboso ma mai mostrati direttamente all'utente.

Configurazioni tipiche per prod:

- \* Angular servito da Spring Boot tramite resources/static;
- \* API disponibili sotto /api/\*\*;
- \* nessun CORS necessario se frontend e backend sono serviti dallo stesso host;
- \* console H2 disabilitata;
- \* logging essenziale e controllato;
- \* nessuna esposizione di dettagli tecnici o stack trace;
- \* configurazioni sensibili lette da variabili d'ambiente o file esterni;
- \* eventuale database locale in modalita' file, se previsto dai requisiti, con percorso documentato.

## Logging per ambiente

Il progetto deve prevedere una configurazione dei log differenziata tra sviluppo e produzione.

Requisiti logging ambiente dev:

- \* livello di log piu' dettagliato per facilitare debug e sviluppo;
- \* possibilita' di abilitare log DEBUG per package applicativi;
- \* log chiari per chiamate REST, errori di validazione, autenticazione e operazioni principali sui contatti;
- \* log tecnici sufficienti a diagnosticare problemi locali;
- \* nessuna esposizione di password, token JWT, hash, segreti o dati sensibili nei log.

Requisiti logging ambiente prod:

- \* livello di log piu' essenziale, preferibilmente INFO o WARN secondo contesto;
- \* log degli errori applicativi e degli eventi rilevanti senza rumore eccessivo;
- \* nessun dettaglio tecnico sensibile nei messaggi restituiti all'utente;
- \* nessun logging di credenziali, token, dati personali non necessari o informazioni riservate;
- \* stack trace completi solo se necessari nei log server, mai nelle risposte REST;
- \* configurazione coerente con application-prod.yml.

Comando sviluppo backend:

```
./gradlew bootRun --args='--spring.profiles.active=dev'
```

Comando produzione backend:

```
java -jar build/libs/rubricaJavaAngular-x.y.z.jar --spring.profiles.active=prod
```

Frontend Angular

Il frontend deve usare environment separati per sviluppo e produzione.

File consigliati:

```
frontend/src/environments/environment.ts  
frontend/src/environments/environment.development.ts  
frontend/src/environments/environment.production.ts
```

Requisiti:

- \* in sviluppo Angular deve chiamare le API backend con URL esplicito, ad esempio `http://localhost:8080/api`;
- \* in produzione Angular deve chiamare le API con path relativo, ad esempio `/api`;
- \* la configurazione dell'API base URL deve stare in environment o configurazione dedicata, non hardcoded nei servizi;
- \* il build Angular di produzione deve usare l'environment production;
- \* eventuale proxy Angular di sviluppo deve essere documentato se usato.

Configurazione sviluppo consigliata:

```
production: false  
apiBaseUrl: http://localhost:8080/api
```

Configurazione produzione consigliata:

```
production: true  
apiBaseUrl: /api
```

Flussi operativi

Sviluppo locale:

1. Avviare backend Spring Boot con profilo dev.
2. Avviare frontend Angular su localhost:4200.
3. Angular chiama Spring Boot su localhost:8080/api.
4. CORS o proxy Angular gestiscono la comunicazione tra porte diverse.

Produzione/integrata:

1. Generare build Angular production.
2. Copiare gli artifact Angular in backend/src/main/resources/static.
3. Generare jar Spring Boot con Gradle.
4. Avviare Spring Boot con profilo prod.
5. L'utente accede al path base del backend.
6. Angular chiama le API tramite path relativo /api.

Questa distinzione deve essere descritta nel README del progetto.

## Repository e pubblicazione

Il codice del progetto dovrà essere pubblicato su GitHub sotto il namespace:

<https://github.com/Michelepal>

Il repository finale dovrà essere creato o individuato all'interno di tale namespace, ad esempio:

<https://github.com/Michelepal/rubricaJavaAngular>

Requisiti per la pubblicazione:

- \* mantenere una struttura repository chiara con frontend/, backend/ e documentazione alla root;
- \* includere un README.md alla root del repository per spiegare il progetto;
- \* nel README.md descrivere obiettivo dell'applicazione, architettura, tecnologie usate, struttura delle cartelle, requisiti di avvio, configurazione locale, comandi per frontend, comandi per backend, test e note sulla sicurezza;
- \* includere il file dei requisiti del progetto;
- \* includere un file con lo storico dei prompt forniti durante la progettazione dei requisiti;
- \* evitare di pubblicare credenziali, token, password, file temporanei, build locali o database locali;
- \* configurare .gitignore per Angular, Gradle, Java, IDE e file generati;
- \* usare commit con messaggi chiari e coerenti;
- \* pubblicare solo codice compilabile, testabile e coerente con i requisiti definiti.

## Versionamento

Ogni commit pubblicato su GitHub deve comportare un aggiornamento della versione del programma, sia lato frontend sia lato backend.

Requisiti di versionamento:

- \* mantenere una versione applicativa frontend, ad esempio nel package.json Angular;
- \* mantenere una versione applicativa backend rispettando gli standard Gradle e Spring Boot;
- \* definire la versione backend tramite la proprietà Gradle standard version nel file build.gradle o build.gradle.kts;
- \* mantenere coordinate Gradle coerenti, includendo almeno group, version e nome progetto in settings.gradle;
- \* evitare meccanismi di versionamento custom che non siano integrati con la build Gradle;
- \* usare la versione Gradle del progetto come fonte principale per artifact, build e packaging Spring Boot;
- \* aggiornare entrambe le versioni prima di ogni commit destinato al repository GitHub;
- \* usare una convenzione di versionamento coerente, preferibilmente Semantic Versioning nel formato MAJOR.MINOR.PATCH;
- \* incrementare PATCH per correzioni e modifiche minori;
- \* incrementare MINOR per nuove funzionalità compatibili;
- \* incrementare MAJOR per modifiche incompatibili o cambi rilevanti;
- \* documentare nel README o in un changelog la modalità scelta per il versionamento;
- \* evitare commit pubblicati senza allineamento delle versioni frontend e backend.

Per il backend Spring Boot:

- \* il nome del progetto deve essere definito in settings.gradle, ad esempio rootProject.name = 'rubricaJavaAngular';
- \* la versione deve essere gestita dalla build Gradle con version = 'x.y.z';
- \* eventuali artifact generati da bootJar o task Gradle devono riflettere la versione definita nella build;
- \* la modifica della versione non deve essere hardcoded in classi Java se non attraverso

configurazione letta dalla build o dalle properties applicative;

\* se la versione viene esposta tramite API, deve derivare da proprietà applicative o build metadata coerenti con Gradle.

## Qualità del codice

Il codice deve rispettare i principi SOLID ed essere progettato per risultare testabile, manutenibile e comprensibile a chi lo legge.

Requisiti generali:

- \* applicare il principio di responsabilità singola, evitando classi o componenti con troppe responsabilità;
- \* separare chiaramente logica di presentazione, logica applicativa, accesso ai dati, sicurezza e validazione;
- \* preferire dipendenze esplicite e facilmente sostituibili nei test;
- \* evitare accoppiamento non necessario tra controller, service, repository, componenti e servizi frontend;
- \* usare interfacce o astrazioni quando aiutano realmente la testabilità o la separazione delle responsabilità;
- \* mantenere metodi e funzioni di dimensione ragionevole;
- \* evitare duplicazioni significative;
- \* evitare codice duplicato applicando il principio DRY, estraendo logica comune in servizi, helper, mapper, validator o componenti riutilizzabili quando questo migliora chiarezza e manutenibilità;
- \* usare nomi chiari, coerenti e descrittivi per classi, metodi, variabili, componenti, servizi, DTO ed entity;
- \* evitare abbreviazioni ambigue e nomi generici come data, item, obj, manager, utility, se non giustificati dal contesto;
- \* inserire commenti dove aiutano a comprendere decisioni, regole di business o passaggi non immediati;
- \* evitare commenti ovvi che ripetono semplicemente cosa fa una singola istruzione;
- \* mantenere il codice leggibile anche per sviluppatori che non hanno partecipato alla progettazione iniziale.

Il codice Java backend e il codice JavaScript/TypeScript frontend devono essere progettati per essere testabili con la copertura massima possibile.

Requisiti di testabilità e copertura:

- \* strutturare codice Java e TypeScript in moduli, classi, funzioni e servizi facilmente testabili in isolamento;
- \* evitare logica complessa direttamente nei controller, nei componenti Angular o nei template;
- \* separare logica pura, accesso dati, chiamate HTTP, validazioni, mapping e gestione stato;
- \* rendere mockabili dipendenze esterne, repository, servizi HTTP, storage locale, sicurezza e provider di tempo;
- \* coprire con test unitari la maggior parte della logica Java e TypeScript;
- \* coprire con test di integrazione i flussi critici di autenticazione, autorizzazione, CRUD e persistenza;
- \* usare strumenti di coverage per misurare la copertura del codice;
- \* per il backend Java, prevedere strumenti compatibili con Gradle come JaCoCo;

- \* per il frontend Angular, prevedere coverage tramite il test runner Angular configurato;
- \* puntare alla copertura piu' alta possibile, con particolare attenzione a regole di business, validazioni, sicurezza, service layer, guard, interceptor, sanitizer e modali;
- \* documentare eventuali parti non coperte e la motivazione;
- \* evitare codice non testabile o fortemente accoppiato se non strettamente necessario;
- \* non sacrificare chiarezza e manutenibilita' solo per aumentare artificialmente la percentuale di coverage.

I test devono generare report sullo stato dell'applicazione.

Requisiti report test:

- \* generare report dei test backend;
- \* generare report dei test frontend;
- \* generare report di coverage backend;
- \* generare report di coverage frontend;
- \* produrre un riepilogo leggibile dello stato applicativo dopo l'esecuzione dei test;
- \* indicare test eseguiti, test superati, test falliti, test saltati e percentuale di copertura;
- \* evidenziare eventuali aree critiche non coperte;
- \* rendere i report consultabili localmente dopo l'esecuzione dei task di test;
- \* documentare nel README dove trovare i report generati;
- \* evitare che report generati e temporanei vengano committati, salvo scelta esplicita documentata.

Report consigliati:

```
backend/build/reports/tests/
backend/build/reports/jacoco/
frontend/coverage/
```

Nel backend Spring Boot:

- \* i controller devono delegare la logica applicativa ai service;
- \* i service devono contenere le regole di business e rimanere testabili con mock dei repository;
- \* i repository devono occuparsi solo dell'accesso ai dati;
- \* la sicurezza deve essere isolata in classi dedicate;
- \* i DTO devono separare il contratto API dalle entity JPA;
- \* mapping e normalizzazione dei dati devono essere collocati in punti chiari e testabili.

Nel frontend Angular:

- \* i componenti devono gestire principalmente stato UI e interazioni utente;
- \* i servizi devono gestire chiamate API, autenticazione e logica condivisa;
- \* validator e sanitizer devono essere funzioni o servizi isolati e testabili;
- \* guard, interceptor e servizi devono avere responsabilita' distinte;
- \* i template devono rimanere leggibili, evitando logica complessa direttamente nell'HTML.

## Frontend Angular



Il frontend deve essere realizzato con Angular e deve includere almeno:

- \* pagina di login;
- \* home dell'applicazione;
- \* pagina di errore;
- \* lista contatti;
- \* form creazione contatto;
- \* form modifica contatto;
- \* servizio autenticazione;
- \* servizio contatti;
- \* interceptor HTTP per inviare il token di autenticazione;
- \* guardia di rotta per proteggere le sezioni riservate.

## Requisiti grafici e responsive

L'interfaccia grafica deve essere responsive e correttamente utilizzabile su tutti i principali breakpoint.

Requisiti:

- \* supportare viewport mobile, tablet, laptop e desktop;
- \* definire breakpoint chiari e coerenti con il layout Angular/CSS scelto;
- \* garantire che menu, liste, form, tabelle, modali e pulsanti rimangano utilizzabili anche su schermi piccoli;
- \* evitare sovrapposizioni tra testi, controlli e contenitori;
- \* evitare scroll orizzontale non necessario;
- \* adattare tabelle e liste a layout mobile leggibili;
- \* mantenere le modali visibili e usabili su viewport piccoli;
- \* usare spaziature, altezze dei controlli e aree cliccabili adeguate anche per uso touch;
- \* ridimensionare il testo quando necessario, mantenendolo leggibile ma mai eccessivamente grande o troppo piccolo;
- \* scegliere un font adeguato alla visibilità e alla lettura prolungata;
- \* usare una scala tipografica coerente per titoli, sottotitoli, etichette, testo dei campi e messaggi di errore;
- \* garantire contrasto sufficiente tra testo e sfondo;
- \* permettere all'utente di passare tra modalità chiara e modalità scura;
- \* mantenere leggibilità, contrasto e coerenza grafica sia in light mode sia in dark mode;
- \* salvare la preferenza del tema dell'utente, ad esempio in local storage o nel profilo utente se previsto;
- \* rispettare, se opportuno, la preferenza di sistema prefers-color-scheme come valore iniziale;
- \* verificare che testi lunghi, email, numeri di telefono e nomi tag non rompano il layout;
- \* mantenere la UI coerente tra login, home, lista contatti, form, modali e pagina errore.

Il font deve essere leggibile e adatto a una applicazione gestionale. Sono preferibili font sans-serif moderni e ben supportati, ad esempio Inter, Segoe UI, Roboto, Arial o equivalenti.

Lo switch tra tema chiaro e tema scuro deve essere disponibile nell'interfaccia utente, preferibilmente nella topbar o nelle impostazioni, e deve applicarsi a tutte le viste dell'applicazione.

## Organizzazione CSS

Il CSS deve essere organizzato in modo da evitare conflitti tra regole globali e regole specifiche delle singole pagine o componenti.

Requisiti:

- \* le regole CSS generiche devono stare in file globali dedicati;
- \* le regole CSS specifiche devono stare nei file stile dei singoli componenti o pagine Angular;
- \* evitare selettori globali troppo ampi che possano alterare componenti non correlati;
- \* usare naming chiaro e coerente per classi CSS;
- \* preferire variabili CSS o tema centralizzato per colori, spaziature, font e breakpoint;
- \* mantenere reset, typography, tema, utility e layout base separati dagli stili specifici di pagina;
- \* evitare duplicazioni di regole CSS;
- \* evitare conflitti tra tema chiaro e tema scuro;
- \* evitare l'uso non controllato di !important;
- \* verificare che modifiche CSS a una pagina non rompano layout o componenti di altre pagine;
- \* documentare la convenzione CSS scelta.

Struttura consigliata:

```
frontend/src/styles/  
base.css  
theme.css  
typography.css  
utilities.css  
responsive.css
```

```
frontend/src/app/<feature>/<component>/<component>.component.css
```

Le regole globali devono contenere solo stili realmente condivisi. Le regole locali devono rimanere vicine al componente Angular a cui appartengono.

## Compatibilit  browser

La grafica deve essere correttamente visibile e utilizzabile nei browser moderni principali.

Browser da supportare:

- \* Google Chrome;
- \* Microsoft Edge;
- \* Mozilla Firefox;
- \* Safari;
- \* Brave;
- \* altri browser moderni basati su Chromium, quando possibile.

## Requisiti:

- \* evitare API CSS o JavaScript sperimentali non supportate dai browser target, salvo fallback;
- \* verificare layout, responsive design, modali, form, tabelle, menu e switch tema sui browser target;
- \* garantire rendering coerente di font, spaziature, colori, focus e stati interattivi;
- \* usare fallback CSS dove necessario;
- \* evitare dipendenze da comportamenti specifici di un singolo browser;
- \* garantire funzionamento corretto di local storage o meccanismo equivalente per il salvataggio del tema;
- \* verificare che validazioni, modali e navigazione da tastiera siano coerenti tra browser;
- \* documentare eventuali limitazioni note.

## Accessibilita' e usabilita'

La grafica deve rispettare criteri di accessibilita' e usabilita' anche per persone diversamente abili.

## Requisiti:

- \* progettare l'interfaccia seguendo le linee guida WCAG, preferibilmente livello AA;
- \* garantire navigazione completa da tastiera;
- \* rendere visibile e chiaro lo stato di focus degli elementi interattivi;
- \* usare elementi HTML semantici dove possibile;
- \* associare correttamente label e descrizioni ai campi form;
- \* usare attributi ARIA solo quando necessari e in modo coerente;
- \* garantire compatibilita' con screen reader per form, errori, modali, menu e azioni principali;
- \* comunicare gli errori di validazione anche in modo testuale, non solo tramite colore;
- \* mantenere contrasto adeguato tra testo, icone, bordi, focus e sfondi;
- \* non affidare informazioni essenziali solo al colore;
- \* prevedere testi chiari per bottoni, azioni distruttive e modali di conferma;
- \* rendere modali accessibili con focus trap, chiusura controllata e ritorno del focus all'elemento di origine;
- \* garantire dimensioni adeguate per aree cliccabili e controlli touch;
- \* evitare animazioni e transizioni che possano disturbare l'utente;
- \* rispettare, dove possibile, la preferenza utente prefers-reduced-motion;
- \* mantenere la UI semplice, prevedibile e coerente tra le viste;
- \* rendere comprensibili icone e pulsanti tramite testo visibile, aria-label o tooltip accessibili;
- \* verificare che tema chiaro e tema scuro rispettino entrambi i requisiti di accessibilita'.

Ogni operazione di modifica o cancellazione dei dati deve passare attraverso una modale di conferma prima dell'invio effettivo della richiesta al backend.

La modale deve essere usata almeno per:

- \* modifica di un contatto;
- \* cancellazione di un contatto;
- \* modifica di telefono, email, indirizzo o tag associati;
- \* cancellazione di telefono, email, indirizzo o tag associati;

- \* eventuale cancellazione dell'account utente, se prevista in futuro.

I tag associati ai contatti devono essere modificabili e cancellabili dall'interfaccia grafica.

La gestione dei tag deve prevedere:

- \* modifica del nome tag;
- \* cancellazione del tag;
- \* validazione del nome tag prima del salvataggio;
- \* controllo di unicità del tag per utente;
- \* modale di conferma prima di ogni modifica;
- \* modale di conferma prima di ogni cancellazione;
- \* nessuna chiamata API se l'utente annulla la modale;
- \* aggiornamento della UI dopo conferma e risposta positiva del backend.

La conferma deve distinguere chiaramente tra:

- \* annullamento dell'operazione, senza chiamata API;
- \* conferma dell'operazione, con invio della chiamata API;
- \* errore durante l'operazione, con messaggio coerente all'utente.

La modale deve evitare conferme ambigue: il testo deve indicare l'entità coinvolta e l'azione richiesta, ad esempio modifica o cancellazione.

## Routing frontend

Rotte richieste:

/ redirect a /login  
/login pagina login  
/home home applicazione, protetta  
/error pagina errore  
/contacts lista contatti, protetta  
/contacts/new  
/contacts/:id/edit

Dal path base localhost:<porta>/ l'utente deve essere reindirizzato a /login.

Dopo login riuscito, l'utente deve essere reindirizzato alla home Angular.

In caso di credenziali errate o errore applicativo, l'utente deve essere reindirizzato alla pagina /error, eventualmente con un parametro che indichi il motivo dell'errore.

Esempi:

/error?reason=login-failed  
/error?reason=unauthorized  
/error?reason=forbidden

/error?reason=server

## Backend Spring Boot

Il backend deve essere un progetto Gradle + Spring Boot.

Dipendenze previste:

- \* Spring Web;
- \* Spring Data JPA;
- \* Spring Security;
- \* Spring Validation;
- \* H2 Database;
- \* supporto JWT;
- \* eventuale Lombok, se coerente con lo stile del progetto.

## Convenzioni Gradle e Spring Boot

Il progetto backend deve rispettare le convenzioni standard di Gradle e Spring Boot per nomenclature, struttura delle cartelle e organizzazione del codice.

Struttura backend prevista:

```
backend/  
build.gradle  
settings.gradle  
gradlew  
gradlew.bat  
src/  
main/  
java/  
<base-package>/  
RubricaApplication.java  
config/  
controller/  
dto/  
entity/  
exception/  
mapper/  
repository/  
security/  
service/  
resources/  
application.yml  
data.sql o import.sql, se necessari  
test/  
java/  
<base-package>/  
controller/
```

repository/  
security/  
service/

Convenzioni richieste:

- \* usare un package base coerente, ad esempio it.example.rubrica o equivalente;
- \* mantenere la classe principale Spring Boot nel package root, ad esempio RubricaApplication;
- \* usare nomi classi descrittivi e allineati agli standard Java:
  - ContactController;
  - ContactService;
  - ContactRepository;
  - ContactRequest;
  - ContactResponse;
  - SecurityConfig;
  - JwtService;
- \* usare suffissi coerenti con il ruolo della classe:
  - Controller per i controller REST;
  - Service per la logica applicativa;
  - Repository per l'accesso dati;
  - Request e Response per i DTO;
  - Config per la configurazione;
  - Exception per eccezioni applicative;
- \* separare DTO ed entity JPA;
- \* non esporre direttamente le entity JPA nelle API REST;
- \* collocare i test in src/test/java rispettando lo stesso package delle classi testate;
- \* usare src/main/resources/application.yml o application.properties per la configurazione;
- \* mantenere configurazioni, sicurezza, controller, service e repository in package dedicati;
- \* usare Gradle Wrapper per permettere l'avvio del progetto senza richiedere una installazione Gradle globale;
- \* rispettare le convenzioni Gradle per task, dipendenze e source set;
- \* evitare nomi generici come Manager, Utility, Data, Object quando non aggiungono significato.

## API richieste

Endpoint pubblici:

POST /api/auth/login  
POST /api/auth/logout

Endpoint protetti:

GET /api/auth/me

GET /api/contacts  
GET /api/contacts/{id}

POST /api/contacts  
PUT /api/contacts/{id}  
DELETE /api/contacts/{id}

GET /api/tags  
POST /api/tags  
PUT /api/tags/{id}  
DELETE /api/tags/{id}

Gli endpoint della rubrica devono essere accessibili solo da utenti autenticati.

Ogni operazione sui contatti deve essere filtrata in base all'utente autenticato, impedendo l'accesso ai dati di altri utenti.

## Gestione errori REST

Le chiamate REST devono gestire correttamente tutti gli errori sia lato backend sia lato frontend, restituendo sempre messaggi comprensibili all'utente.

Requisiti backend:

- \* definire una gestione centralizzata degli errori, ad esempio con `@ControllerAdvice` e `@ExceptionHandler`;
- \* restituire risposte di errore strutturate e coerenti;
- \* includere informazioni utili come timestamp, status HTTP, codice errore applicativo, messaggio leggibile e path della richiesta;
- \* non esporre stack trace, dettagli interni, query SQL, token, password o informazioni sensibili;
- \* gestire errori di validazione DTO con messaggi specifici per campo;
- \* gestire errori di autenticazione e autorizzazione con 401 e 403;
- \* gestire risorse non trovate con 404;
- \* gestire conflitti, duplicati e vincoli univoci con 409;
- \* gestire payload non validi con 400;
- \* gestire errori imprevisti con 500 e messaggio generico comprensibile;
- \* mantenere codici HTTP coerenti con il tipo di errore;
- \* loggare internamente gli errori tecnici senza mostrarli all'utente.

Formato errore consigliato:

```
{  
  "timestamp": "2026-05-08T12:00:00Z",  
  "status": 400,  
  "error": "VALIDATION_ERROR",  
  "message": "Alcuni campi non sono validi.",  
  "path": "/api/contacts",  
  "fieldErrors": {  
    "firstName": "Il nome e' obbligatorio."  
  }  
}
```

## Requisiti frontend:

- \* intercettare gli errori HTTP tramite servizio dedicato o HTTP interceptor;
- \* mostrare messaggi comprensibili e non tecnici all'utente;
- \* associare gli errori di validazione ai rispettivi campi form quando possibile;
- \* gestire errori 400, 401, 403, 404, 409, 500 e assenza di connessione;
- \* reindirizzare a /login o /error quando la sessione non e' valida o il login fallisce;
- \* mostrare messaggi coerenti in modali, form, toast o pagina errore;
- \* evitare di mostrare dettagli tecnici come stack trace o nomi interni delle classi;
- \* prevedere fallback generico quando il backend non restituisce un errore strutturato;
- \* mantenere i messaggi accessibili anche per screen reader.

## Autenticazione e sicurezza

Il login deve essere gestito con Spring Security e Spring Boot.

Flusso previsto:

Utente apre /

-> redirect a /login

Utente invia credenziali

-> Angular chiama POST /api/auth/login

Spring Security autentica l'utente

-> se OK, il backend restituisce un token JWT

-> Angular salva il token e naviga a /home

-> se KO, Angular naviga a /error

## Requisiti sicurezza backend

- \* Usare AuthenticationManager per validare le credenziali.
- \* Usare UserDetailsService per caricare gli utenti.
- \* Salvare le password con hash BCrypt.
- \* Non salvare mai password in chiaro.
- \* Usare JWT per autenticare le richieste successive.
- \* Configurare un filtro JWT prima di UsernamePasswordAuthenticationFilter.
- \* Proteggere tutti gli endpoint /api/contacts/\*\*.
- \* Consentire pubblicamente solo gli endpoint di login e le risorse statiche necessarie.
- \* Configurare CORS per permettere le chiamate dal frontend Angular in sviluppo.
- \* Restituire codici HTTP coerenti:
  - 401 Unauthorized per credenziali errate;
  - 403 Forbidden per utente non autorizzato o disabilitato;
  - 500 Internal Server Error per errori applicativi non gestiti.

## Database locale

Il database locale consigliato per la base progetto e' H2 in file mode.



La struttura deve supportare:

- \* utenti;
- \* contatti;
- \* numeri di telefono multipli;
- \* email multiple;
- \* indirizzi multipli;
- \* tag/categorie per organizzare la rubrica.

## Modello dati

### Tabella users

id BIGINT PK  
username VARCHAR(50) NOT NULL UNIQUE  
email VARCHAR(254) NOT NULL UNIQUE  
password\_hash VARCHAR(255) NOT NULL  
first\_name VARCHAR(80)  
last\_name VARCHAR(80)  
enabled BOOLEAN NOT NULL DEFAULT TRUE  
created\_at TIMESTAMP NOT NULL  
updated\_at TIMESTAMP NOT NULL

### Tabella contacts

id BIGINT PK  
user\_id BIGINT NOT NULL FK -> users.id  
first\_name VARCHAR(80) NOT NULL  
last\_name VARCHAR(80)  
company VARCHAR(120)  
job\_title VARCHAR(120)  
notes VARCHAR(1000)  
favorite BOOLEAN NOT NULL DEFAULT FALSE  
created\_at TIMESTAMP NOT NULL  
updated\_at TIMESTAMP NOT NULL

### Tabella contact\_phones

id BIGINT PK  
contact\_id BIGINT NOT NULL FK -> contacts.id  
type VARCHAR(30) NOT NULL  
phone\_number VARCHAR(30) NOT NULL  
primary\_phone BOOLEAN NOT NULL DEFAULT FALSE

## Tabella contact\_emails

id BIGINT PK  
contact\_id BIGINT NOT NULL FK -> contacts.id  
type VARCHAR(30) NOT NULL  
email VARCHAR(254) NOT NULL  
primary\_email BOOLEAN NOT NULL DEFAULT FALSE

## Tabella contact\_addresses

id BIGINT PK  
contact\_id BIGINT NOT NULL FK -> contacts.id  
type VARCHAR(30) NOT NULL  
street VARCHAR(160)  
city VARCHAR(100)  
province VARCHAR(80)  
postal\_code VARCHAR(20)  
country VARCHAR(80)  
primary\_address BOOLEAN NOT NULL DEFAULT FALSE

## Tabella tags

id BIGINT PK  
user\_id BIGINT NOT NULL FK -> users.id  
name VARCHAR(40) NOT NULL  
color VARCHAR(20)

UNIQUE(user\_id, name)

## Tabella contact\_tags

contact\_id BIGINT NOT NULL FK -> contacts.id  
tag\_id BIGINT NOT NULL FK -> tags.id

PRIMARY KEY(contact\_id, tag\_id)

## Relazioni

users 1 -> N contacts  
contacts 1 -> N contact\_phones  
contacts 1 -> N contact\_emails  
contacts 1 -> N contact\_addresses  
users 1 -> N tags  
contacts N -> N tags tramite contact\_tags

Ogni contatto appartiene a un solo utente.

Ogni tag appartiene a un solo utente.

Un contatto puo' avere piu' telefoni, email, indirizzi e tag.

## Regole di cancellazione

- \* Eliminando un utente, devono essere eliminati i suoi contatti, tag e dati collegati.
- \* Eliminando un contatto, devono essere eliminati telefoni, email, indirizzi e associazioni ai tag.
- \* Eliminando un tag, devono essere eliminate le associazioni in contact\_tags.

## Validazione e sanitizzazione frontend

Il frontend deve usare Reactive Forms con validator standard e custom.

## Campi contatto

firstName:

- \* obbligatorio
- \* minimo 2 caratteri
- \* massimo 80 caratteri
- \* ammessi lettere, spazi, apostrofi e trattini

lastName:

- \* massimo 80 caratteri
- \* ammessi lettere, spazi, apostrofi e trattini

company:

- \* massimo 120 caratteri

jobTitle:

- \* massimo 120 caratteri

notes:

- \* massimo 1000 caratteri
- \* nessun HTML attivo

favorite:

- \* boolean

## Campi telefono

phoneNumber:

- \* obbligatorio
- \* massimo 30 caratteri
- \* ammessi numeri, +, spazi, trattini, parentesi e punti

## Campi email

email:

- \* obbligatoria
- \* formato email valido
- \* massimo 254 caratteri
- \* normalizzazione lowercase

## Campi indirizzo

street: massimo 160 caratteri  
city: massimo 100 caratteri  
province: massimo 80 caratteri  
postalCode: massimo 20 caratteri  
country: massimo 80 caratteri

## Campi tag

name:

- \* obbligatorio
- \* massimo 40 caratteri
- \* univoco per utente
- \* nessun HTML attivo

## Validator Angular custom

Validator previsti:

- \* personNameValidator;
- \* phoneValidator;
- \* safeTextValidator;

- \* tagNameValidator;
- \* noOnlyWhitespaceValidator.

Sanitizzazione lato frontend:

- \* trim dei campi testuali;
- \* collasso degli spazi multipli;
- \* rimozione dei tag HTML dai campi liberi;
- \* normalizzazione lowercase delle email;
- \* normalizzazione dei tipi di telefono, email, indirizzo e tag.

La sanitizzazione frontend migliora l'esperienza utente, ma non deve essere considerata un controllo di sicurezza sufficiente.

## Validazione backend

Il backend deve validare i DTO in ingresso usando Bean Validation.

Esempio di regole:

firstName:

- \* @NotBlank
- \* @Size(min = 2, max = 80)
- \* @Pattern per nomi validi

email:

- \* @NotBlank
- \* @Email
- \* @Size(max = 254)

phoneNumber:

- \* @NotBlank
- \* @Size(max = 30)
- \* @Pattern per formato telefonico

notes:

- \* @Size(max = 1000)

Il backend deve inoltre:

- \* eseguire trim e normalizzazione dei dati;
- \* sanificare i campi liberi;
- \* verificare che la risorsa richiesta appartenga all'utente autenticato;
- \* impedire duplicati dove previsto;
- \* gestire il vincolo di un solo telefono, email o indirizzo principale per contatto;
- \* usare DTO separati dalle entity JPA;
- \* restituire errori di validazione strutturati e leggibili dal frontend.

## Vincoli database

Il database deve includere:

- \* chiavi primarie;
- \* chiavi esterne;
- \* NOT NULL sui campi obbligatori;
- \* UNIQUE su username ed email utente;
- \* UNIQUE(user\_id, name) sui tag;
- \* lunghezze massime coerenti con i DTO;
- \* indici su campi utili alla ricerca e ai join.

Regole come "un solo telefono principale per contatto" possono essere gestite nel service layer, soprattutto usando H2, dove gli indici parziali non sono la scelta piu' portabile.

## Test unitari frontend

I test Angular devono coprire almeno:

### Validator custom

- \* personNameValidator accetta nomi validi.
- \* personNameValidator rifiuta stringhe vuote, solo spazi, numeri e caratteri non ammessi.
- \* phoneValidator accetta numeri locali e internazionali validi.
- \* phoneValidator rifiuta numeri troppo corti, troppo lunghi o con caratteri non ammessi.
- \* safeTextValidator rifiuta input con tag HTML o script.
- \* tagNameValidator rifiuta tag vuoti, troppo lunghi o con HTML.
- \* noOnlyWhitespaceValidator rifiuta stringhe composte solo da spazi.

### AuthService

- \* invia le credenziali a POST /api/auth/login.
- \* salva il token dopo login riuscito.
- \* rimuove il token al logout.
- \* riconosce correttamente lo stato autenticato/non autenticato.
- \* gestisce errore 401 senza salvare token.

### AuthGuard

- \* consente accesso a /home se il token e' presente e valido.
- \* reindirizza a /login se il token manca.
- \* reindirizza a /error o /login in caso di token non valido o scaduto.

### HTTP interceptor

- \* aggiunge header Authorization: Bearer <token> alle richieste protette.
- \* non aggiunge il token alla richiesta di login.
- \* gestisce risposte 401 o 403 secondo la strategia definita.

## Componenti

- \* LoginComponent mostra errori di validazione sui campi obbligatori.
- \* LoginComponent naviga a /home in caso di login riuscito.
- \* LoginComponent naviga a /error in caso di login fallito.
- \* ContactFormComponent impedisce submit con dati non validi.
- \* ContactFormComponent apre una modale di conferma prima di inviare una modifica.
- \* ContactFormComponent non invia la richiesta di modifica se la modale viene annullata.
- \* ContactFormComponent normalizza i dati prima dell'invio.
- \* ContactsListComponent mostra la lista ricevuta dal servizio.
- \* ContactsListComponent apre una modale di conferma prima di cancellare un contatto.
- \* ContactsListComponent non invia la richiesta di cancellazione se la modale viene annullata.
- \* la modale di conferma mostra testo coerente con l'azione richiesta.
- \* la modale di conferma chiama il servizio corretto solo dopo conferma esplicita.
- \* i tag associati ai contatti sono modificabili tramite interfaccia dedicata.
- \* la modifica di un tag apre una modale di conferma prima della chiamata API.
- \* la cancellazione di un tag apre una modale di conferma prima della chiamata API.
- \* annullando la modale di modifica o cancellazione tag non viene inviata alcuna chiamata API.
- \* confermando la modale di modifica o cancellazione tag viene chiamato il servizio corretto.

## Responsive UI

- \* layout login correttamente visibile su mobile, tablet e desktop.
- \* layout home correttamente visibile su mobile, tablet e desktop.
- \* lista contatti leggibile senza scroll orizzontale non necessario.
- \* form contatto utilizzabile su schermi piccoli.
- \* modale di conferma centrata e usabile su mobile.
- \* testi lunghi non sovrapposti ad altri elementi.
- \* font leggibile e coerente tra le diverse viste.
- \* dimensione del testo adeguata ai diversi breakpoint.
- \* pulsanti e controlli hanno area cliccabile adeguata anche su touch.
- \* switch tema chiaro/scuro disponibile nell'interfaccia.
- \* tema chiaro applicato correttamente a login, home, form, modali e pagina errore.
- \* tema scuro applicato correttamente a login, home, form, modali e pagina errore.
- \* preferenza tema mantenuta dopo refresh o nuova apertura dell'applicazione.
- \* contrasto testi/sfondi adeguato in entrambe le modalita'.

## CSS

- \* regole globali separate da regole specifiche di componente.
- \* assenza di conflitti CSS tra pagine e componenti.
- \* assenza di selettori globali troppo invasivi.
- \* tema chiaro e scuro non introducono conflitti di stile.
- \* modifiche CSS locali non alterano viste non correlate.
- \* convenzione CSS documentata.

## Accessibilita'

- \* tutti i campi form hanno label associate correttamente.
- \* gli errori di validazione sono annunciabili e leggibili.
- \* la navigazione principale e' usabile da tastiera.
- \* il focus e' visibile su pulsanti, link, campi e controlli.
- \* le modali gestiscono correttamente focus iniziale, focus trap e ritorno del focus.
- \* i bottoni iconici hanno nome accessibile.
- \* il contrasto e' adeguato in modalita' chiara e scura.
- \* le azioni distruttive sono comprensibili anche senza affidarsi solo al colore.
- \* l'interfaccia resta utilizzabile con screen reader.
- \* eventuali animazioni rispettano prefers-reduced-motion.

## Compatibilita' browser

- \* login visibile e utilizzabile nei browser moderni principali.
- \* home visibile e utilizzabile nei browser moderni principali.
- \* form, modali, tabelle e liste mantengono layout coerente tra browser.
- \* switch tema chiaro/scuro funziona tra browser supportati.
- \* local storage o meccanismo di persistenza tema funziona tra browser supportati.
- \* focus, hover e stati disabilitati sono visibili in modo coerente.
- \* nessuna funzionalita' critica dipende da API sperimentali senza fallback.

## Copertura frontend JavaScript/TypeScript

- \* configurazione test Angular per generare report di coverage.
- \* copertura elevata per servizi, validator, sanitizer, guard, interceptor e componenti con logica.
- \* test dei template e delle interazioni utente dove rilevante.
- \* test delle modali di conferma e dello switch tema.
- \* esclusione motivata solo per codice generato o parti prive di logica.
- \* build o task dedicato per eseguire test e report coverage.
- \* report finale con stato test frontend e copertura.

## Test unitari backend

I test backend devono coprire almeno:

## Servizi autenticazione

- \* login riuscito con credenziali valide.
- \* login fallito con password errata.
- \* login fallito con username inesistente.
- \* login fallito con utente disabilitato.
- \* generazione token JWT con subject corretto.
- \* validazione token JWT valido.
- \* rifiuto token JWT scaduto o malformato.

## UserDetailsService



- \* carica un utente esistente.
- \* solleva errore per utente inesistente.
- \* restituisce correttamente enabled/disabled.

## Copertura backend Java

- \* configurazione Gradle per generare report di coverage, preferibilmente con JaCoCo.
- \* copertura elevata per service, security, validator, mapper e controller.
- \* esclusione motivata solo per codice generato, configurazioni banali o classi prive di logica.
- \* build o task dedicato per eseguire test e report coverage.
- \* report finale con stato test backend e copertura.

## Servizi contatti

- \* crea un contatto associandolo all'utente autenticato.
- \* aggiorna solo contatti appartenenti all'utente autenticato.
- \* impedisce aggiornamento di contatti appartenenti ad altri utenti.
- \* elimina solo contatti appartenenti all'utente autenticato.
- \* restituisce solo i contatti dell'utente autenticato.
- \* gestisce correttamente telefono/email/indirizzo principale.
- \* modifica un tag appartenente all'utente autenticato.
- \* impedisce la modifica di tag appartenenti ad altri utenti.
- \* cancella un tag appartenente all'utente autenticato.
- \* impedisce la cancellazione di tag appartenenti ad altri utenti.
- \* normalizza email e campi testuali.
- \* rifiuta dati non validi.

## Validator DTO

- \* rifiuta firstName vuoto o troppo corto.
- \* rifiuta email non valida.
- \* rifiuta telefono non valido.
- \* rifiuta note troppo lunghe.
- \* rifiuta tag vuoti o troppo lunghi.

## Controller REST

- \* POST /api/auth/login restituisce 200 e token con credenziali valide.
- \* POST /api/auth/login restituisce 401 con credenziali errate.
- \* endpoint contatti restituiscono 401 senza token.
- \* endpoint contatti restituiscono 403 se l'utente non e' autorizzato.
- \* GET /api/contacts restituisce solo i contatti dell'utente autenticato.
- \* POST /api/contacts restituisce 400 con payload non valido.
- \* PUT /api/contacts/{id} restituisce 404 se il contatto non esiste per l'utente corrente.
- \* DELETE /api/contacts/{id} elimina il contatto corretto.
- \* endpoint di modifica tag restituisce 400 con nome tag non valido.
- \* endpoint di modifica tag restituisce 404 se il tag non appartiene all'utente corrente.

- \* endpoint di cancellazione tag elimina solo tag appartenenti all'utente autenticato.
- \* errori REST restituiscono formato strutturato e coerente.
- \* errori di validazione restituiscono messaggi specifici per campo.
- \* errori imprevisti non espongono dettagli tecnici o sensibili.

## Gestione errori frontend

- \* interceptor o servizio errori intercetta risposte HTTP non riuscite.
- \* errore 400 mostra messaggi di validazione comprensibili.
- \* errore 401 gestisce sessione non valida o credenziali errate.
- \* errore 403 mostra messaggio di autorizzazione negata.
- \* errore 404 mostra messaggio di risorsa non trovata.
- \* errore 409 mostra messaggio di conflitto o duplicato.
- \* errore 500 mostra messaggio generico comprensibile.
- \* errore di rete mostra messaggio di connessione non disponibile.
- \* messaggi errore sono visibili e accessibili all'utente.

## Repository e database

- \* vincolo unique su username.
- \* vincolo unique su email utente.
- \* vincolo unique su (user\_id, name) per i tag.
- \* cancellazione a cascata dei dati collegati al contatto.
- \* persistenza corretta di telefoni, email, indirizzi e tag.

## Test di integrazione consigliati

Oltre ai test unitari, sono consigliati test di integrazione con Spring Boot Test e database H2:

- \* login end-to-end tramite API;
- \* chiamata protetta con JWT valido;
- \* chiamata protetta con JWT mancante o invalido;
- \* creazione contatto completa con telefoni, email, indirizzi e tag;
- \* verifica isolamento dati tra utenti diversi;
- \* verifica risposta strutturata per errori di validazione.

## Dati iniziali

Per facilitare sviluppo e test, il backend puo' inizializzare un utente demo:

username: admin  
password: admin  
email: admin@example.local

La password deve essere salvata nel database solo in formato hash BCrypt.

Si possono aggiungere alcuni contatti demo associati all'utente admin.

## Criteri di accettazione

- \* Aprendo il path base dell'applicazione, l'utente viene mandato a /login.
- \* Il login e' gestito da Spring Security.
- \* Con credenziali corrette, l'utente arriva alla home Angular.
- \* Con credenziali errate, l'utente arriva alla pagina di errore o visualizza un errore coerente.
- \* Le API della rubrica sono protette.
- \* Ogni utente vede solo i propri contatti.
- \* I dati sono validati lato frontend.
- \* I dati sono validati lato backend.
- \* Il database applica vincoli coerenti con il modello.
- \* Sono presenti test unitari per validator, autenticazione, servizi, controller e repository.
- \* Il progetto puo' essere avviato localmente con frontend Angular e backend Spring Boot.